Java

Portabilité : Java est "Write Once, Run Anywhere" (WORA) grâce à la JVM (Java Virtual Machine). Un programme Java peut s’exécuter sur n’importe quel système (Windows, Linux, macOS) sans modification.

Popularité : Utilisé dans les applications d’entreprise, le développement Android, les systèmes embarqués, le Big Data (Hadoop, Spark), etc.

Sécurité et robustesse : Gestion automatique de la mémoire (garbage collector), vérifications strictes à la compilation et à l’exécution.

- Exemple concret

Android (applications mobiles), Minecraft (jeu vidéo), systèmes bancaires, etc.

Java

- Orienté Objet
- Simple, Robuste, Dynamique et Sécurisé
- Indépendant de la Plateforme (VM)
- Semi Compilé/Semi Interprété
- Fortement type
- Bibliothèque Importante (JDK API)

Bycote

- **Le code source à l’exécution du JVM**

```
Code source (.java)
        ↓ (javac)
Bytecode (.class)
        ↓ (java)
Exécution par la JVM
```

Installation de l’environnement

le JDK : Télécharger version du JDK depuis Oracle ou OpenJDK.

Choisir un IDE :

- IntelliJ IDEA

- **https://www.jetbrains.com/fr-fr/idea/download/?section=windows**

- Eclipse ou VS Code (avec l’extension "Java Extension Pack").

Structure Java

- **Structure d’un programme Java**

```java
// Déclaration de la classe (doit porter le même nom que le fichier)
public class HelloWorld {
    // Méthode principale (point d'entrée du programme)
    public static void main(String[] args) {
        // Instruction pour afficher du texte dans la console
        System.out.println("Hello, World!");
    }
}
```

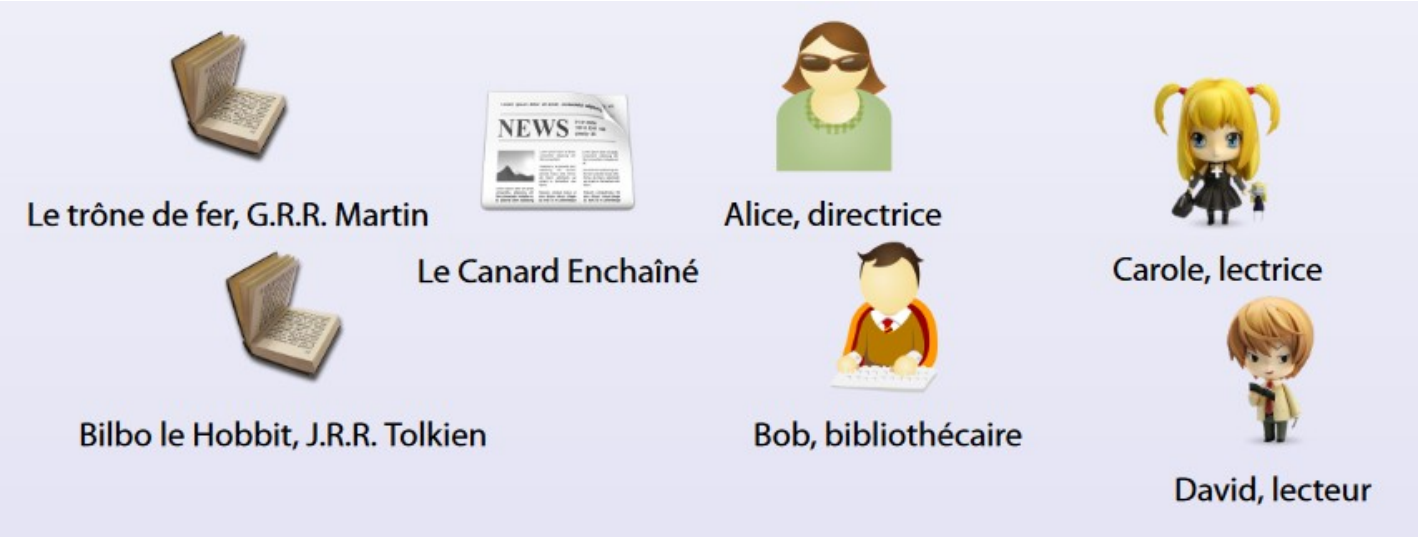
PROGRAMMATION ORIENTÉE OBJET

	IMPERATIF	FONCTIONNEL	OBJET	DESCRIPTIF
DEMARCHE	Procédurale Série d'instructions, sauts conditionnels	Flots de données Diagramme de structure	Objets, classes, composition, réseau de messages	Besoin, expressif, léger
CONCEPTS	Itération, structures de contrôles Exécution d'instructions qui modifient l'état de la mémoire	Evaluation d'expressions qui ne dépendent que de la valeur des arguments, et non de l'état de la mémoire	Objet, classes, méthodes, encapsulation, héritage, relations de composition, d'utilisation, …	Description des buts à atteindre à l'aide d'une syntaxe légère
LANGAGES	Fortran, C, Pascal	Lisp, Scheme, Caml	SmallTalk, C++, Java, Python	HTML, XML, LaTeX

PROGRAMMATION ORIENTÉE OBJET

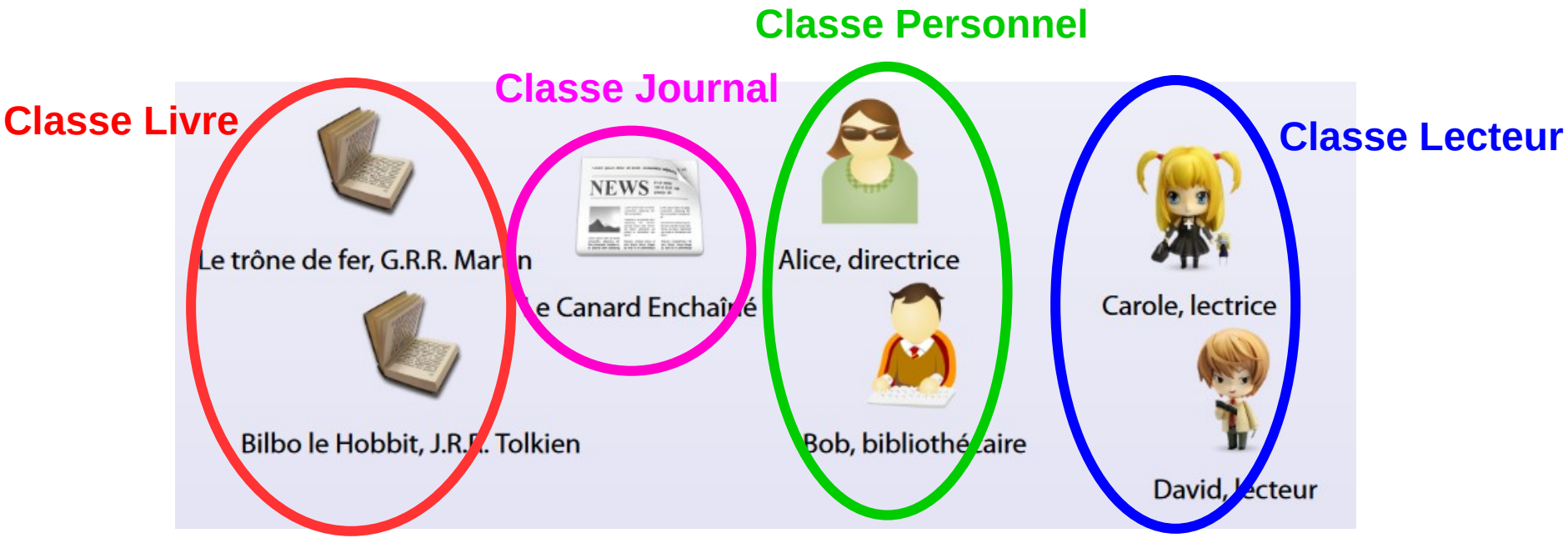
Approche procédurale : “que doit faire mon programme ?

Approche objet : “de quoi doit être composé mon programme ?”



PROGRAMMATION ORIENTÉE OBJET

Approche objet : “de quoi doit être composé mon programme ?”



Règles de nommage en Java

1. Conventions générales

En Java, le respect des conventions de nommage améliore la lisibilité et la maintenance du code. Voelles-ci sont largement adoptées par la communauté des développeurs.

2. Noms de classes et interfaces

Format : PascalCase (ou UpperCamelCase) Chaque mot commence par une majuscule, sans séparateur. Exemple : VoiciUnNomDeClasse CompteBancaire GestionnaireUtilisateurs

3. Noms de méthodes et variables

Format : camelCase Le premier mot commence par une minuscule, chaque mot suivant commence par une majuscule. Exemples : calculerSomme() nombreEtudiants estValide

4. Constantes (variables statiques finales)

Format : MAJUSCULES_AVEC_UNDERSCORE Tous les caractères sont en majuscules, les mots sont séparés par des underscores (_). Exemple : Integer.MAX_VALUE TAUX_TVA_STANDARD

Règles de nommage en Java

5. Autres éléments

Paquets (packages) : En minuscules, sans underscore. Exemple : com.example.monprojet

Paramètres de méthodes : camelCase, comme les variables. Exemple : public void ajouterEtudiant(String nomEtudiant)

6. Bonnes pratiques supplémentaires

Éviter les caractères spéciaux (sauf _ pour les constantes).

Éviter les abréviations obscures : Sauf si elles sont largement comprises (ex : id, url).

7. Pourquoi ces conventions ?

Uniformité : Facilite la collaboration entre développeurs.

Lisibilité : Rend le code plus compréhensible et auto-documenté.

Compatibilité : Respecte les standards de la plateforme Java et des outils d’analyse de code.

Remarque : Ces règles sont définies dans le Java Code Conventions d’Oracle.

Classe - Objet

Qu’est-ce qu’une classe ?

Une classe est un modèle qui définit :

- Les attributs (ou champs) : les données que l’objet possédera.
- Les méthodes : les actions (fonctions) que l’objet pourra effectuer.

Analogie : Une classe est comme un plan de construction pour une maison.

Elle décrit ce que la maison (l’objet) aura (portes, fenêtres) et ce qu’elle pourra faire (ouvrir une porte, fermer une fenêtre).

```java
public class Voiture {
    // Attributs (données)
    String couleur;
    int anneeFabrication;
    double vitesse;

    // Méthodes (actions)
    void accelerer(double increment) {
        vitesse += increment;
    }

    void afficherVitesse() {
        System.out.println("Vitesse actuelle : " + vitesse + " km/h");
    }
}
```

Classe - Objet

Attributs vs Méthodes

Attributs	Méthodes
Stockent des données.	Contiennent du code pour effectuer des opérations.
Représentent les propriétés ou l’état d’un objet.	Représentent les comportements ou actions de l’objet.
Exemple : couleur, anneeFabrication, vitesse.	Exemple : accelerer(), afficherVitesse().

Classe - Objet

Classe	Objet
Modèle abstrait (ex : le plan d’une voiture).	Instance concrète (ex : une voiture rouge de 2020).
Définie une fois, utilisée pour créer des objets.	Créé à partir d’une classe, existe en mémoire.
N’occupe pas de place en mémoire.	Occupe de la mémoire (allouée lors de la création).

Classe - Objet

Instancier une classe

Instancier signifie créer un objet à partir d'une classe, en utilisant le mot-clé new. Cela alloue de la mémoire pour stocker les données de l'objet.

- Pour utiliser concrètement les fonctionnalités définies dans la classe.
- Chaque objet a ses propres valeurs d'attributs (ex : une voiture rouge, une autre bleue).

```java
public class Main {
    public static void main(String[] args) {
        // Instanciation : création d'un objet de type Voiture
        Voiture maVoiture = new Voiture();
        maVoiture.couleur = "Rouge";
        maVoiture.anneeFabrication = 2020;

        // Utilisation des méthodes
        maVoiture.accelerer(50.0);
        maVoiture.afficherVitesse(); // Affiche : "Vitesse actuelle : 50.0 km/h"
    }
}
```

Avantages de la POO

Modularité : Le code est organisé en objets indépendants et autonomes.

Réutilisabilité : Les classes et objets peuvent être réutilisés dans d'autres projets ou contextes.

Maintenance facilitée : Le code POO est plus facile à maintenir et à étendre grâce à l'héritage et l'encapsulation.

Facilité de compréhension : Le modèle d'organisation basé sur les objets reflète souvent la structure du monde réel.

L’héritage en POO : Fondements et Enjeux

Principe de base

L’héritage est l’un des piliers de la Programmation Orientée Objet (POO). Il permet :

- D’organiser les classes de manière hiérarchique (arbre de classes).
- De définir des sous-types pour modéliser des relations de spécialisation.

Vocabulaire clé :

Une classe B hérite d’une classe A → B est une sous-classe (ou classe dérivée) de A.

A est appelée superclasse (ou classe parente).

Les objets de B sont aussi des instances de A (relation "is a" : "B est un A").

Avantages de l’héritage

- Réutilisation de code : Factorisation des attributs et méthodes communs.
- Conception progressive : Spécialisation des classes existantes sans tout réécrire.
- Polymorphisme : Possibilité de traiter des objets de sous-classes de manière uniforme.

Exemple : Si A est Vehicule et B est Voiture, alors Voiture est un Vehicule.

Voiture hérite des attributs (marque, modele) et méthodes (demarrer(), arreter()) de Vehicule.

L’héritage en POO : Fondements et Enjeux

Mécanisme d’héritage en Java

Héritage simple : Une classe ne peut hériter que d’une seule superclasse (pas d’héritage multiple).

Accès aux membres : La classe dérivée hérite des attributs et méthodes public et protected de la superclasse. Les membres private ne sont pas accessibles directement.

```
public class Voiture extends Vehicule {
    // Voiture hérite des membres public/protected de Vehicule
    // et peut en ajouter de nouveaux.
}
```

L’héritage en POO : Fondements et Enjeux

Classes abstraites

Définition : Une classe abstraite est une classe qui ne peut pas être instanciée directement. Elle sert de modèle partiel pour ses sous-classes.

Utilité :

- Définir un contrat : Obliger les sous-classes à implémenter certaines méthodes (méthodes abstraites).
- actoriser du code commun : Éviter la duplication dans les sous-classes.

Pourquoi ?

- Imposer une structure commune (ex : tous les véhicules doivent avoir une méthode demarrer()).
- Centraliser la logique partagée (ex : gestion de la marque).

```java
public abstract class Vehicule {
    protected String marque;

    // Méthode abstraite : doit être implémentée par les sous-classes
    public abstract void demarrer();

    // Méthode concrète : réutilisable par les sous-classes
    public void afficherMarque() {
        System.out.println("Marque : " + marque);
    }
}

public class Voiture extends Vehicule {
    @Override
    public void demarrer() {
        System.out.println("La voiture démarre avec une clé.");
    }
}
```

L’héritage en POO : Fondements et Enjeux

Interfaces

Définition : Une interface est un contrat qui définit un ensemble de méthodes (sans implémentation), que les classes doivent respecter.

Utilité :

- Définir des comportements transverses : Une classe peut implémenter plusieurs interfaces (contrairement à l’héritage simple).
- Favoriser le découplage : Permet de dissocier la définition d’un comportement de son implémentation.

```java
public interface Conducible {
    void accelerer(double vitesse);
    void freiner();
}

public class Voiture implements Conducible {
    @Override
    public void accelerer(double vitesse) {
        System.out.println("Accélération à " + vitesse + " km/h");
    }

    @Override
    public void freiner() {
        System.out.println("Freinage...");
    }
}
```

L’héritage en POO : Fondements et Enjeux

Héritage (extends)	Interface (implements)
Relation "is a" (spécialisation).	Relation "can do" (capacité).
Partage de code et d’état (attributs).	Définition de comportements sans implémentation.
Une seule superclasse possible.	Une classe peut implémenter plusieurs interfaces.

Composition et Agrégation

Principe Commun

La composition et l’agrégation modélisent une relation d’appartenance du type "**has a" ou "is part of"** entre deux classes.

Elles permettent de construire des objets complexes à partir d’objets plus simples, en évitant l’héritage (relation "is a").

Comment Identifier une Association ?

Une classe A est en association avec une classe B si B apparaît dans A sous l’une de ces formes :

Attribut de A (ex : private B b;).

Paramètre d’une méthode de A (ex : void methode(B b)).

Type de retour d’une méthode de A (ex : B getB()).

Objet créé ou utilisé dans le corps d’une méthode de A.

Composition et Agrégation

Critère	Composition	Agrégation
Relation de dépendance	Forte : B ne peut exister sans A.	Faible : B peut exister indépendamment de A.
Cycle de vie	B est créé et détruit avec A.	B existe avant et après A.
Exemple	Une Voiture a un Moteur.	Un Département a des Employés.

Composition et Agrégation

Composition (Relation forte)

```
public class Voiture {
    private Moteur moteur; // Le moteur n'existe pas sans la voiture

    public Voiture() {
        this.moteur = new Moteur(); // Créé avec la voiture
    }
}
```

Si la Voiture est détruite, le Moteur n'a plus de raison d'exister.

Agrégation (Relation faible)

```
public class Departement {
    private List<Employe> employes; // Les employés existent indépendamment

    public Departement(List<Employe> employes) {
        this.employes = employes; // Passés en paramètre (existent déjà)
    }
}
```

Les Employes peuvent appartenir à d'autres départements ou exister seuls.

Association Composition et Agrégation

Avantages

Modularité : Le code est plus flexible et maintenable.

Réutilisabilité : Les composants (B) peuvent être réutilisés dans d’autres contextes.

Évite l’héritage : Préférez la composition à l’héritage pour des relations dynamiques ou complexes.

Les Collections

Présentation du Framework

Le Framework Collections en Java est une architecture unifiée conçue pour :

- Stocker et manipuler des groupes d’objets de manière efficace.
- Généraliser et étendre la notion de tableau (plus flexible, dynamique, et riche en fonctionnalités).

Principales Interfaces et Leur Rôle

Le framework propose plusieurs interfaces de base, chacune répondant à un besoin spécifique :

Les Collections

Interface	Description	Cas d’Usage Typique
List	Collection ordonnée (éléments accessibles par index). Autorise les doublons.	Listes de tâches, historiques, séquences.
Set	Collection sans doublons. L’ordre n’est pas garanti (sauf pour LinkedHashSet).	Ensembles uniques (ex : identifiants, tags).
Queue	Collection FIFO (First In, First Out) ou prioritaire.	Files d’attente, gestion de tâches.
Map	Stocke des paires clé-valeur. Les clés sont uniques.	Dictionnaires, caches, configurations.

Les Collections

List

ArrayList : Tableau dynamique (accès rapide par index).

LinkedList : Liste doublement chaînée (insertions/suppressions rapides).

Vector : Similaire à ArrayList, mais synchronisé (thread-safe).

Set

HashSet : Basé sur une table de hachage (accès rapide, ordre non garanti).

TreeSet : Trie les éléments (implémente SortedSet).

LinkedHashSet : Maintient l’ordre d’insertion.

Queue

PriorityQueue : File prioritaire (éléments triés selon un comparateur).LinkedList : Implémente aussi List et Deque (file double).

Map

HashMap : Table de hachage (accès rapide par clé).

TreeMap : Trie les entrées par clé.

Hashtable : Version synchronisée (obsolète au profit de ConcurrentHashMap).